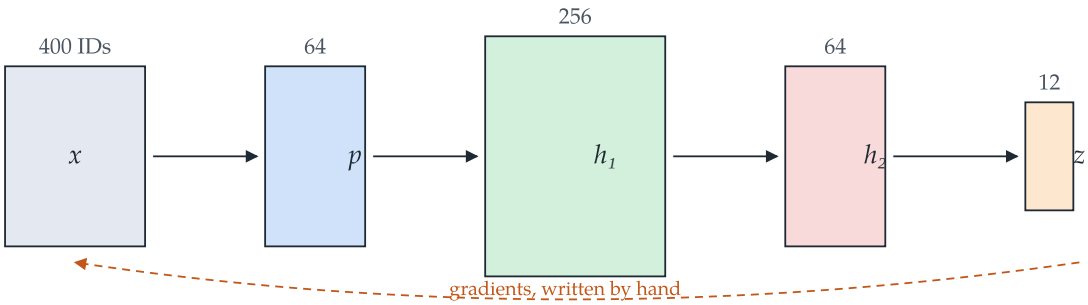
# An MLP Built from Scratch, One Complaint at a Time

A shape-by-shape report on the `complaint-triage-mlp` repository




Lasfer Mohammed Djawed

`github.com/Fyl3x/complaint-triage-mlp`

October 2026

This document follows one consumer complaint through every step of the project: tokens and IDs, the embedding lookup and the sum, each layer of the MLP, the logits and the loss, the hand-written backward pass, the AdamW update, and the PyTorch rebuild that checks it. Every tensor is given with its shape, and the worked numbers were produced by the repository's own `src/` code: the published epoch-3 checkpoint was reproduced from the same seed (validation loss equal to $10^{-11}$, identical test accuracy 74.54%) and probed. Results and figures come from `results/`.

# Contents

**1 The whole project on one page 3**
1.1 What the project is 3
1.2 Pipeline at a glance 3
1.3 Code map 3
1.4 The configuration every number in this document assumes 4
1.5 Notation and colours 4
1.6 Where the 1.46 million parameters are 4

**2 Data: from the CFPB export to three splits 5**
2.1 Source and label merging 5
2.2 Cap, split, and the resulting class sizes 5
2.3 What the text looks like 6

**3 One complaint, from text to a vector 7**
3.1 Tokenisation and the vocabulary 7
3.2 Encoding: IDs, unknown words, padding 8
3.3 Batching 8
3.4 The embedding lookup 9
3.5 What padding does to the sum 10

**4 Through the MLP head, layer by layer 11**
4.1 The linear layer and how shapes align 12
4.2 Batch normalisation 13
4.3 ReLU and dropout 14
4.4 The output layer and the logits 15
4.5 The example, layer by layer (eval mode) 15

**5 From logits to a loss 16**
5.1 Softmax, computed stably 16
5.2 Label smoothing 16
5.3 The loss of the example 17

**6 Backpropagation by hand 18**
6.1 The rule that organises everything 18
6.2 Softmax + cross-entropy: the gradient on the logits 18
6.3 Linear layer 19
6.4 Dropout and ReLU 20
6.5 Batch normalisation 20
6.6 The embedding: a scatter-add 21
6.7 One real step: the gradient of every parameter 22
6.8 Checking the hand-written gradients 22

**7 Learning: AdamW and the training loop 24**
7.1 AdamW 24
7.2 The training loop 25
7.3 Initialisation and the first loss 26

**8 NumPy versus PyTorch: what autograd replaces 27**
8.1 The same network in about thirty lines 27
8.2 Conventions, side by side 28
8.3 What "the two match" means in this project 29

**9 Evaluation and results 31**
9.1 Metrics 31
9.2 Headline results 31
9.3 Training curves 31
9.4 Per-class behaviour and the label overlap 32
9.5 The TF-IDF + logistic regression baseline 33
9.6 The checkpoint bug 34

**10 Findings, limitations and conclusion 36**
10.1 What the project shows 36
10.2 Limitations 36
10.3 Next experiments (a possible v2) 36
10.4 Conclusion 36

**A Shape cheat sheet 37**

**B How this document was verified 37**

# 1 The whole project on one page

## 1.1 What the project is

The task is to read a consumer's complaint about a financial product, as filed with the US Consumer Financial Protection Bureau (CFPB), and decide which of 12 product categories it belongs to: *Mortgage*, *Student loan*, *Credit reporting* and so on. The model is a multilayer perceptron (MLP) on top of a summed word embedding. The point of the project is not the model's size but its construction. In **Phase 1** every layer, every backward pass, the loss and the optimizer are written by hand in NumPy, with no automatic differentiation. In **Phase 2** the same network is rebuilt in PyTorch, where autograd computes the gradients, and is used as a reference for Phase 1. The two reach their best validation loss at the same epoch and score **74.54%** and **74.10%** on the held-out test set. A TF-IDF + logistic-regression baseline scores 77.24%. Chapter 9 explains why.

## 1.2 Pipeline at a glance

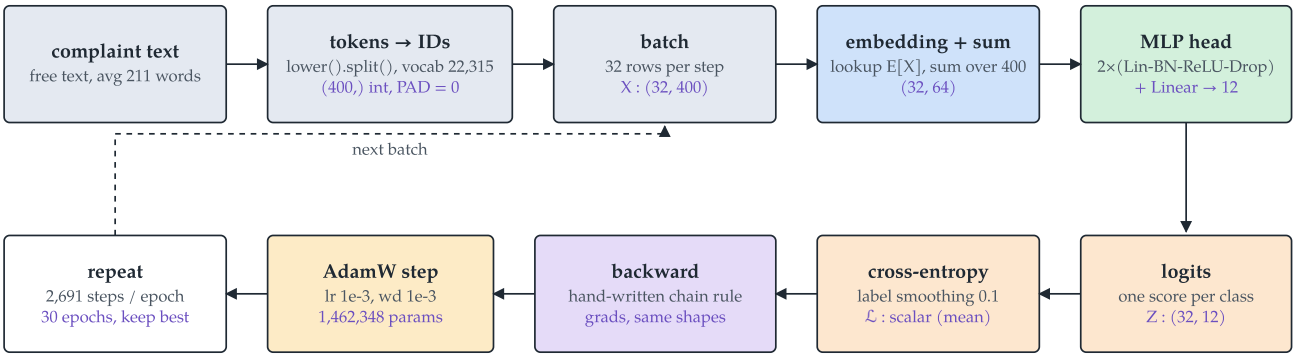



**Figure 1.** One training step. The top row is the forward pass; the bottom row is the loss, the backward pass and the update. Purple text is a tensor shape, here with batch size 32.

## 1.3 Code map

| File | What it contains | Chapter |
|---|---|---|
| `src/vocab.py` | `Vocab`: counting, `word2idx`, `encode` (pad/truncate to 400) | 3 |
| `src/embedding.py` | `Embedding`: lookup + sum, scatter-add backward | 3, 6 |
| `src/linear.py`, `batchnorm.py`, `relu.py`, `dropout.py` | the layers, each with `forward` and a hand-written `backward` | 4, 6 |
| `src/mlp.py` | `MLPHead`: wiring of the 9 layers, reverse-order `backward` | 4, 6 |
| `src/crossentropy.py` | stable softmax, label smoothing 0.1, backward | 5, 6 |
| `src/optimizer.py` | AdamW (bias correction, decoupled weight decay) | 7 |
| `src/backend.py` | NumPy or CuPy behind the name `np` | 7 |
| `train_numpy.py` | Phase 1: training loop, checkpoint, test evaluation | 7, 9 |
| `train_torch.py` | Phase 2: same network with `nn.Module` + autograd | 8 |
| `baseline_tfidf.py` | TF-IDF + logistic regression baseline | 9 |
| `src/prepare_data.py` | label merging, per-class cap, 64/16/20 split | 2 |

## 1.4 The configuration every number in this document assumes

| Quantity | Value | Where it is set |
|---|---|---|
| Classes | 12 product categories | `prepare_data.py` |
| Vocabulary | 22,315 IDs (words seen ≥ 10 times in train, plus <PAD>=0, <UNK>=1) | `Vocab(10, 400, …)` |
| Sequence length | 400 tokens (pad or truncate) | `max_len=400` |
| Embedding | 64 dimensions, init $N(0, 0.01^2)$, summed over positions | `Embedding(V, 64)` |
| Head | 64 → 256 → 64 → 12, BatchNorm + ReLU + Dropout 0.5 | `MLPHead` |
| Loss | cross-entropy, label smoothing 0.1, mean over batch | `CrossEntropyLoss` |
| Optimizer | AdamW, lr $10^{-3}$, weight decay $10^{-3}$, $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$ | `Optimizer` |
| Batch / epochs | 32 / 30 (2,691 steps per epoch) | `train_numpy.py` |
| Checkpoint | lowest validation loss (epoch 3 in both phases) | `train_numpy.py` |
| Seed | 42 (NumPy in Phase 1, `torch.manual_seed` in Phase 2) | both scripts |

## 1.5 Notation and colours

$B$ is the batch size (32 in training), $T = 400$ the sequence length, $V = 22{,}315$ the vocabulary size, $d = 64$ the embedding width and $K = 12$ the number of classes. For any tensor $X$ in the computation, $G_X = \partial\mathcal{L}/\partial X$ is the gradient of the scalar loss with respect to it; it **always has the shape of X**. Shapes follow NumPy order, for example $(B, 256)$. Matrices in the diagrams are drawn as boxes with their row count on the left and their column count on top. Box colours follow the stage: token IDs | embedding (64) | hidden (256) | hidden (64) | logits / loss | weights | gradients.

The boxes separate **general ideas** (the mathematics), **what this repository does** (with sizes), **details and deviations** (where the code differs from a textbook or could surprise a reader), and **shape checks**.

## 1.6 Where the 1.46 million parameters are

| Tensor | Shape | Count |
|---|---|---|
| Embedding $E$ | $(22315, 64)$ | 1,428,160 |
| Linear1 $W_1$ (no bias) | $(64, 256)$ | 16,384 |
| BatchNorm1 $\gamma$, $\beta$ | $(256,) \times 2$ | 512 |
| Linear2 $W_2$ (no bias) | $(256, 64)$ | 16,384 |
| BatchNorm2 $\gamma$, $\beta$ | $(64,) \times 2$ | 128 |
| Linear3 $W_3$, $b$ | $(64, 12)$, $(1, 12)$ | 780 |
| **Total trainable** | | **1,462,348** |

98% of the parameters are the embedding table. The MLP head itself has only 34,188. The two BatchNorm layers also keep $2 \times (256 + 64) = 640$ running statistics that are *not* trained and are *not* returned by `parameters()`. That detail becomes the checkpoint bug of Section 9.6.

# 2 Data: from the CFPB export to three splits

## 2.1 Source and label merging

The input is the public CFPB Consumer Complaint Database export (`complaints.csv`, 9.2 GB). `src/prepare_data.py` keeps only rows with a consumer-written narrative and unifies product names that the CFPB changed over the years:

| Merged label | Raw CFPB product names mapped to it |
|---|---|
| Credit reporting | Credit reporting; Credit reporting, credit repair services, or other personal consumer reports; Credit reporting or other personal consumer reports |
| Credit card | Credit card; Credit card or prepaid card |
| Bank account or service | Bank account or service; Checking or savings account |
| Payday/title/personal loan | Payday loan; Payday loan, title loan, or personal loan; Payday loan, title loan, personal loan, or advance loan |
| Money transfer / virtual currency | Money transfers; Money transfer, virtual currency, or money service; Virtual currency |
| *dropped* | Other financial service |

**Detail / deviation**

Not every renamed category is merged. *Consumer Loan* (an older category) and *Vehicle loan or lease* / *Payday/title/personal loan* (the newer ones that cover much of the same ground) stay separate classes, and so does *Debt or credit management*, which overlaps with credit repair complaints that the first merge put into *Credit reporting*. Section 9.4 shows that these are exactly the classes the models confuse.

## 2.2 Cap, split, and the resulting class sizes

```
15 data3 = data[data["Product"] ≠ "Other financial service"]
16 data4 = data3.groupby("Product").apply(lambda x: x.sample(n=min(len(x),12000),random_state=42)).
   reset_index(drop=False)
17 data5 = data4.drop(columns=["level_1"])
18
19 training, test = train_test_split(data5,test_size=0.2,train_size=0.8,stratify=data5["Product"],r
   andom_state=42)
20 train, val = train_test_split(training,test_size=0.2,train_size=0.8,stratify=training["Produc
   t"],random_state=42)
```

src/prepare_data.py: per-class cap and the two stratified splits

| Class | Rows | |
|---|---|---|
| Mortgage | 12,000 | |
| Credit reporting | 12,000 | |
| Bank account or service | 12,000 | |
| Debt collection | 12,000 | |
| Student loan | 12,000 | |
| Money transfer / virtual currency | 12,000 | |
| Credit card | 12,000 | |
| Vehicle loan or lease | 12,000 | |
| Payday/title/personal loan | 12,000 | |
| Prepaid card | 11,635 | below the cap |
| Consumer Loan | 9,458 | below the cap |
| Debt or credit management | 5,492 | below the cap |

Each class is randomly downsampled to at most 12,000 complaints (no oversampling), leaving **134,585** rows. Three classes stay below the cap. Two stratified splits with `random_state=42` give 80/20 and then 80/20 of the remainder, that is 64 / 16 / 20%:

| Split | Rows | Used for |
|---|---|---|
| train | 86,134 | gradient steps, vocabulary |
| val | 21,534 | checkpoint choice, tuning |
| test | 26,917 | reported once |

Labels are mapped to integers by sorting the 12 names alphabetically (`dictio`), so class 0 is *Bank account or service* and class 11 is *Vehicle loan or lease*.

## 2.3 What the text looks like

**In this repository**

Measured on the training split with the project's own tokenizer: complaints are **210.7 words** long on average (median 146, 90th percentile 434, longest 5,858). **11.9%** are longer than 400 tokens and get truncated; the other 88% are padded. On average 222 of the 400 positions are <PAD>, so **55.5%** of all positions in a batch are padding. The CFPB redaction token `xxxx` (dates, names, amounts) is the second most frequent token after *the*, with 654,218 occurrences. 1.18% of validation words are out of vocabulary (<UNK>).

# 3 One complaint, from text to a vector

From here on we follow one real complaint from the test split (row 95, true label **Mortgage**), which the trained Phase 1 model classifies correctly:

> *"Company did not pay taxes on time since XXXX 2021 and cost me hundreds in not thousands in interest and penalties. Further, they created an escrow shortage that significantly increased my payment amount"*

## 3.1 Tokenisation and the vocabulary

```
        self.count = {} # this is a counter that has {word : # of Occurences}
        for text in self.texts:
            for w in text.split():
                w = w.lower() # lower case to not mix up anything
                self.count[w] = self.count.get(w,0) + 1 # basic dictionary method to increase co
unt
        # self.count gives us the word and how many times it occured in the narratives
        # now word2idx uses counts to see which qualifies to be a token in itself (count ⩾ min_
count)
        self.word2idx = {}
        self.word2idx["<UNK>"] = 1 # Special CHAR
        self.word2idx["<PAD>"] = 0 # Special CHAR
        i = 2 # Start count at 0
        for key,val in self.count.items():
            if val ⩾ self.min_count:
                self.word2idx[key] = i
                i+=1
```

src/vocab.py: counting words over the training texts and assigning IDs

**General idea**

A neural network needs numbers, so every distinct word gets an integer ID. The vocabulary is built from the **training split only**: if it saw validation or test text, the model would get information about data it is later evaluated on.

**In this repository**

Tokenisation is `text.split()` after `lower()`: whitespace splitting, nothing else. The training texts contain 126,600 distinct lower-cased words; the 22,313 that occur at least 10 times get IDs 2, 3, ... in order of first appearance, and two special IDs are reserved: **<PAD> = 0** and **<UNK> = 1**. So $V = 22{,}315$.

**Detail / deviation**

Punctuation stays attached to words: `loan`, `loan.`, `loan,` and `loans.` are four different IDs. In the example, `penalties.` (ID 5678) and `further,` (ID 3044) are tokens of their own. The TF-IDF baseline's tokenizer strips punctuation, which is part of its advantage (Section 9.5).

## 3.2 Encoding: IDs, unknown words, padding

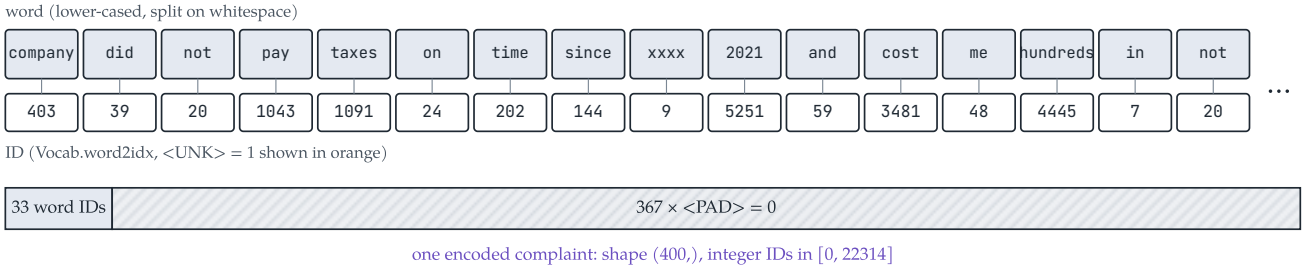



**Figure 2.** The first 16 of the example's 33 words and their IDs. None are out of vocabulary. The encoded sequence is always 400 long: 33 word IDs, then 367 <PAD> IDs.

All 33 word→ID pairs: `company`→403, `did`→39, `not`→20, `pay`→1043, `taxes`→1091, `on`→24, `time`→202, `since`→144, `xxxx`→9, `2021`→5251, `and`→59, `cost`→3481, `me`→48, `hundreds`→4445, `in`→7, `not`→20, `thousands`→5125, `in`→7, `interest`→439, `and`→59, `penalties.`→5678, `further,`→3044, `they`→150, `created`→3245, `an`→107, `escrow`→95, `shortage`→108, `that`→115, `significantly`→442, `increased`→2117, `my`→2, `payment`→60, `amount`→756.

```
40      def encode(self,text):
41          text_list = self.translate(text)
42          result = text_list.copy()
43          if len(text_list) > self.max_len:
44              result = text_list[:self.max_len]
45          elif len(text_list) < self.max_len:
46              for w in range(self.max_len-len(text_list)):
47                  result.append(self.word2idx["<PAD>"])
48          else:
49              result = result
50          return result
```

src/vocab.py: encode() truncates to max_len or appends PAD (0) up to max_len

## 3.3 Batching

All texts are encoded once, before training (`Whole_Vocab` in `train_numpy.py`, shape (86134, 400)). Each step slices 32 rows by a shuffled index list:

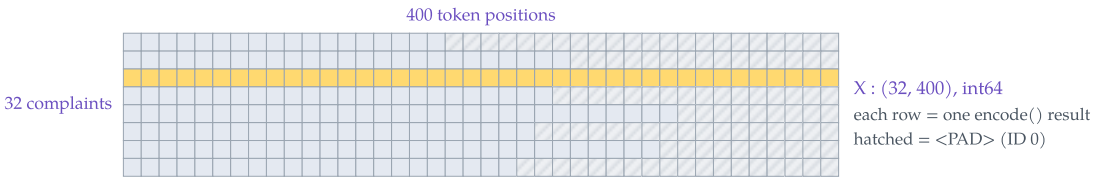



**Figure 3.** The model input for one step is an integer matrix. Rows are complaints, columns are token positions.

## 3.4 The embedding lookup

```python
class Embedding(Module):
    def __init__(self,vocab_size,embed_dim):
        self.vocab_size = vocab_size
        self.embed_dim = embed_dim
        self.weight = np.random.randn(self.vocab_size,self.embed_dim) * 0.01

    def forward(self,batch):
        self.batch = batch
        result = self.weight[batch] # this gives B,T, embed_dim
        result_summed = result.sum(axis = -2) # this is now B,C
        return result_summed
```

src/embedding.py: the whole forward pass is one fancy-index and one sum

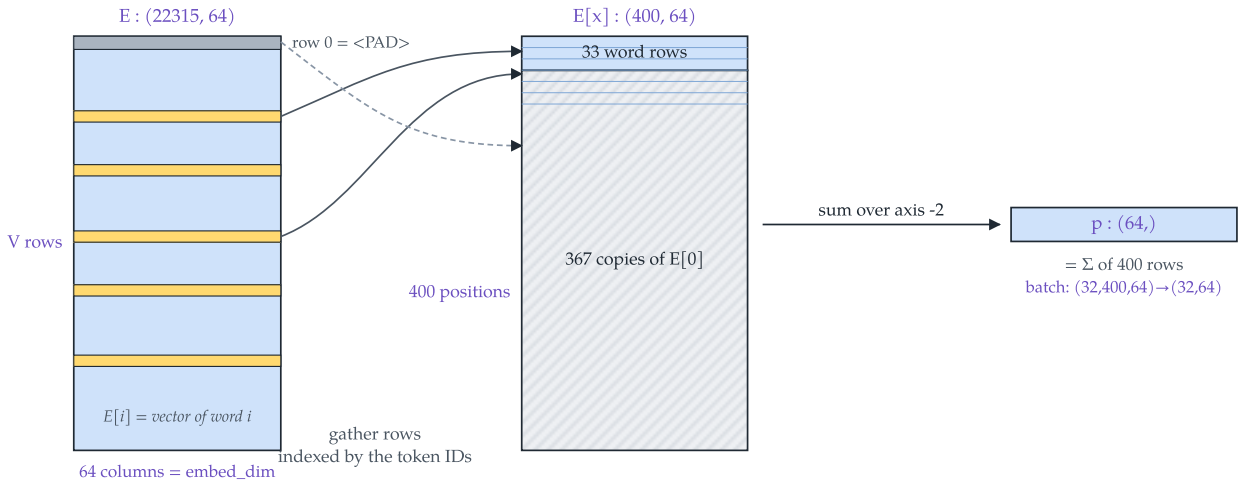



**Figure 4.** `self.weight[batch]` picks one 64-number row of $E$ per token. For the example that is 33 distinct word rows and 367 copies of row 0. Summing over the 400 positions gives a fixed-size vector, whatever the text's length.

**General idea**

An embedding is a learned lookup table $E \in \mathbb{R}^{V \times d}$: row $i$ is the vector for word $i$. Looking up a row is the same as multiplying a one-hot vector by $E$, so it is a linear layer whose input happens to be an index. Summing the rows of a sentence gives a *bag-of-words* representation: word order is lost, and only which words occur (and how often) remains.

$$p_n = \sum_{t=1}^{400} E[x_{n,t}] \in \mathbb{R}^{64}, \qquad P = (p_1; \ldots; p_B) \in \mathbb{R}^{B \times 64}.$$

**Shape check**

`batch` (B, 400) int → `self.weight[batch]` (B, 400, 64) → `.sum(axis=-2)` (B, 64). For one validation chunk of 1,024 complaints the intermediate tensor is 1024·400·64·8 bytes = 210 MB, which is why evaluation runs in chunks (`predict_logits`).

## 3.5 What padding does to the sum

**In this repository: the PAD row on the example**

Row 0 is a trainable vector like any other, and nothing masks it out. In the trained model it is small: $\|E[0]\| = 0.053$, against an average row norm of 0.616. But the example adds it 367 times. Splitting the pooled vector for the example:

$$p = \underbrace{\textstyle\sum_{33\ \text{words}} E[x_t]}_{\|\cdot\| = 3.18} + \underbrace{367 \cdot E[0]}_{\|\cdot\| = 19.45}, \qquad \|p\| = 18.76.$$

The padding part is six times larger than the part that comes from the words. Over the whole test split, the padding part is the larger of the two for **57.6%** of complaints (median ratio 1.29). The pooled vector therefore carries the complaint's length as strongly as its words.

**Detail / deviation**

The consequence is visible in the results: accuracy on the shortest quarter of test complaints ($\leq$ 78 words, most padding) is **66.2%**, against 77.3%, 76.7% and 78.2% for the other three quarters. Short texts also simply contain less evidence, so this is a correlation, not a proof. Masking the padding (or mean-pooling over real tokens only) is the first experiment listed in Chapter 10.

# 4 Through the MLP head, layer by layer

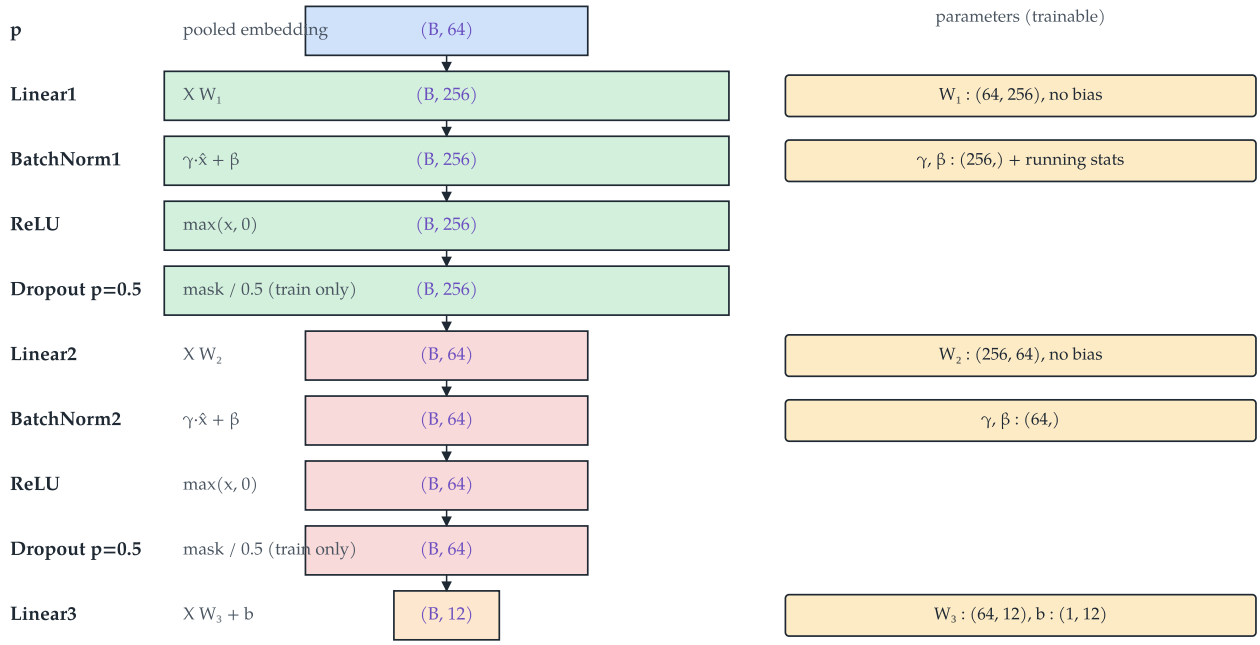



**Figure 5.** `MLPHead.forward`: nine modules in a fixed order. Bar width is proportional to the feature dimension; the amber boxes are the trainable tensors of each layer.

```
7     def __init__(self,embed_dim,output_dim):
8         super().__init__()
9         input_dim = embed_dim
10        out_dim = output_dim
11        self.linear1 = Linear(input_dim,4*input_dim,bias=False)
12        self.bn1 = BatchNorm1d(4*input_dim,momentum=0.7)
13        self.relu1 = ReLU()
14        self.dropout1 = Dropout(percentage=0.5)
15        self.linear2 = Linear(4*input_dim,input_dim,bias=False)
16        self.bn2 = BatchNorm1d(input_dim,momentum=0.7)
17        self.relu2 = ReLU()
18        self.dropout2 = Dropout(percentage=0.5)
19        self.linear3 = Linear(input_dim,out_dim)
20
21    def forward(self,x):
22        x = self.linear1(x)
23        x = self.bn1(x)
24        x = self.relu1(x)
25        x = self.dropout1(x)
26        x = self.linear2(x)
27        x = self.bn2(x)
28        x = self.relu2(x)
29        x = self.dropout2(x)
30        x = self.linear3(x)
31        self.x = x
32        return x
```

src/mlp.py: the head is built and wired by hand; 4*input_dim = 256

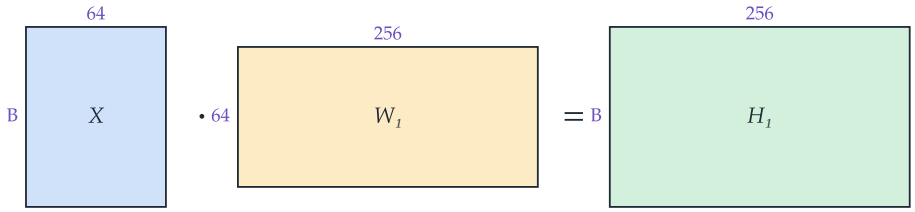
## 4.1 The linear layer and how shapes align

**General idea**

A linear layer maps each input row $x \in \mathbb{R}^{d_{\text{in}}}$ to $xW + b$. For a whole batch this is one matrix product. The inner dimensions must agree, and they disappear:

$$\underbrace{X}_{(B,64)}\ \underbrace{W_1}_{(64,256)} = \underbrace{H_1}_{(B,256)}, \qquad H_1[n,j] = \sum_{i=1}^{64} X[n,i]\, W_1[i,j].$$

Each of the 256 outputs is a weighted sum of all 64 inputs. Column $j$ of $W_1$ is the pattern that output feature $j$ responds to.




Linear1 forward: (B, 64)·(64, 256) = (B, 256). No bias: the BatchNorm right after it would cancel any constant shift.

**Figure 6.** Matrix shapes in the first linear layer.

```python
class Linear(Module):
    def __init__(self,in_dim,out_dim,bias=True):
        self.in_dim = in_dim
        self.has_bias = bias
        self.out_dim = out_dim
        self.weight = np.random.randn(self.in_dim,self.out_dim) * np.sqrt(2/(self.in_dim))
        if bias == True:
            self.bias = np.zeros((1,self.out_dim))
    def forward(self,batch):
        self.batch = batch
        out = batch @ self.weight # this should be (B,out_dim)
        if self.has_bias:
            out += self.bias
        return out
```

src/linear.py: weights stored (in_dim, out_dim) and He-normal initialised: std = sqrt(2 / in_dim)

**Detail / deviation**

The weight is stored as `(in_dim, out_dim)` and applied as `batch @ self.weight`. PyTorch's `nn.Linear` stores the transpose, `(out, in)`, and computes `x @ W.T`. The bias here has shape `(1, out)` and broadcasts over the batch; PyTorch's is `(out,)`. Same function, different layouts. This matters only when weights are copied between the two phases (Chapter 8).

## 4.2 Batch normalisation

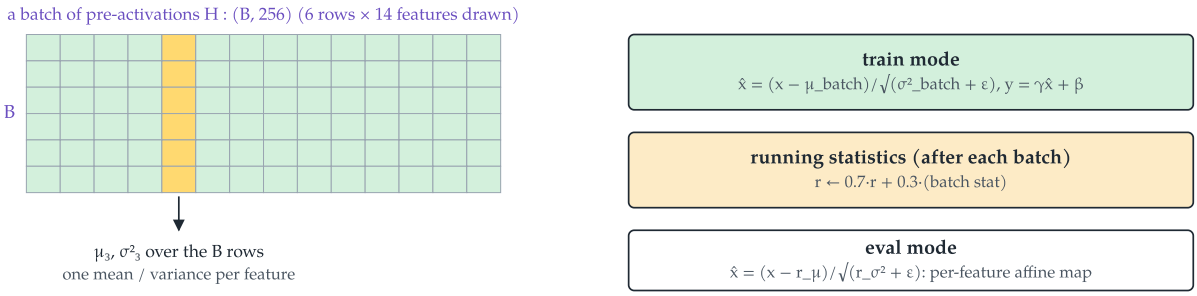



**Figure 7.** BatchNorm1d works per feature (per column): statistics are taken over the batch rows. Training uses the batch's own statistics and updates a running average; evaluation uses only the running average.

```
15      def forward(self,x):
16          if self.training:
17              self.x = x
18              self.loc_mean = x.mean(axis=0,keepdims=True)
19              self.loc_var = x.var(axis=0,keepdims=True)
20              self.running_var = self.momentum*self.running_var + (1-self.momentum)*self.loc_var
21              self.running_mean = self.momentum*self.running_mean + (1-self.momentum)*self.loc_mea
    n
22              mean,var = self.loc_mean,self.loc_var
23          else:
24              mean,var = self.running_mean, self.running_var
25          self.x_norm = (x - mean)/ np.sqrt(var + self.eps)
26
27          result = self.gamma * self.x_norm + self.beta
28          return result
```

src/batchnorm.py: forward in train and eval mode

**General idea**

For each feature $j$ of a batch $H \in \mathbb{R}^{B\times C}$:

$$\mu_j = \tfrac{1}{B}\textstyle\sum_n H_{nj}, \quad \sigma_j^2 = \tfrac{1}{B}\textstyle\sum_n (H_{nj}-\mu_j)^2, \quad \hat{H}_{nj} = \frac{H_{nj}-\mu_j}{\sqrt{\sigma_j^2+\varepsilon}}, \quad Y_{nj} = \gamma_j \hat{H}_{nj} + \beta_j.$$

Every feature enters the ReLU with mean 0 and variance 1 (then rescaled by the learned $\gamma$, $\beta$), whatever the scale of the layer before. In evaluation a single example has no batch, so the running averages $r_\mu$, $r_{\sigma^2}$ are used instead, and the layer becomes a fixed affine map per feature: $y = a_j x + b_j$ with $a_j = \gamma_j/\sqrt{r_{\sigma^2,j}+\varepsilon}$, $b_j = \beta_j - a_j r_{\mu,j}$.

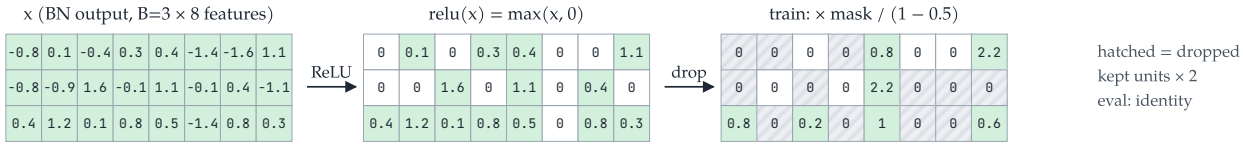
**In this repository: BatchNorm1 of the trained model**

`MLPHead` builds both BatchNorms with `momentum=0.7`: after every training batch, `running = 0.7·running + 0.3·batch_stat`. The variance is the biased one (`x.var()`, divide by $B$), $\varepsilon = 10^{-5}$, $\gamma$ starts at 1 and $\beta$ at 0. The running statistics are plain attributes, re-created as new arrays at every batch.

| $j$ | $\gamma_j$ | $\beta_j$ | $r_\mu$ | $r_{\sigma^2}$ | $a_j$ | $b_j$ |
|---|---|---|---|---|---|---|
| 0 | 0.974 | -0.206 | 0.741 | 15.11 | 0.250 | -0.391 |
| 1 | 1.042 | -0.262 | 7.278 | 20.78 | 0.229 | -1.925 |
| 2 | 1.013 | -0.131 | 3.698 | 31.60 | 0.180 | -0.798 |
| 3 | 0.859 | -0.110 | 6.629 | 22.42 | 0.181 | -1.313 |

BatchNorm1 in the trained model, first four features: eval mode is y = a·x + b

## 4.3 ReLU and dropout




ReLU zeroes negatives (and their gradient); dropout zeroes a fresh random half in training and doubles the rest

**Figure 8.** ReLU keeps positive values. Dropout (training only) zeroes each unit with probability 0.5 and doubles the survivors, so the expected activation is unchanged.

```
    def forward(self,x):
        self.out = np.maximum(x,0)
        self.mask = (x > 0)
        return self.out
    def backward(self,local_grad):
        self.local_grad = local_grad
        # drelu = np.zeros_like(self.out)
        # drelu[self.mask] = local_grad
        drelu = local_grad * self.mask
        return drelu
```

src/relu.py

```
    def forward(self,x):
        result = x
        if self.training:
            s1,s2 = x.shape
            holder_arr = np.random.rand(s1,s2)
            self.mask = holder_arr > self.p
            result = (x * self.mask) / (1-self.p)
        return result


    def backward(self,upstream_grad):
        result = upstream_grad
        if self.training:
            result = upstream_grad * self.mask/(1-self.p)
        return result
```

src/dropout.py: the mask is drawn in forward and reused in backward

**General idea**

**ReLU**, $\max(x, 0)$, is what makes the network non-linear: without it, the three linear layers would collapse into one matrix. **Inverted dropout** multiplies by a random mask $m \sim \text{Bernoulli}(1-p)$ and divides by $1-p$: $\mathbb{E}[m\,x/(1-p)] = x$, so nothing needs rescaling at test time. With $p = 0.5$ each training step uses a random half of the hidden units, which discourages units from depending on each other.

## 4.4 The output layer and the logits

The second block repeats the pattern at width 64 ($W_2$ (256, 64), BatchNorm2, ReLU, Dropout). The last layer has a bias and no normalisation: $Z = H_2 W_3 + b$, (B, 64)·(64, 12) + (1, 12) = (B, 12). Row $n$ of $Z$ holds 12 *logits*, unnormalised scores, one per class.

## 4.5 The example, layer by layer (eval mode)

| after | shape | mean | std | min | max | % zero |
|---|---|---|---|---|---|---|
| Linear1 | (1, 256) | 4.257 | 3.370 | -10.54 | 12.43 | – |
| BatchNorm1 | (1, 256) | 0.196 | 0.396 | -0.86 | 1.15 | – |
| ReLU | (1, 256) | 0.275 | 0.290 | 0.00 | 1.15 | 30.1 |
| Dropout (eval) | (1, 256) | 0.275 | 0.290 | 0.00 | 1.15 | – |
| Linear2 | (1, 64) | -2.522 | 1.660 | -6.56 | 1.13 | – |
| BatchNorm2 | (1, 64) | 0.088 | 0.457 | -0.81 | 1.15 | – |
| ReLU | (1, 64) | 0.236 | 0.315 | 0.00 | 1.15 | 48.4 |
| Dropout (eval) | (1, 64) | 0.236 | 0.315 | 0.00 | 1.15 | – |
| Linear3 (logits) | (1, 12) | -0.342 | 0.927 | -1.32 | 2.40 | – |

Activations of the example complaint in the trained Phase 1 model (BatchNorm on running statistics, dropout off).

**In this repository: what the numbers show**

Linear1 turns the pooled vector (norm 18.76) into 256 values of mean 4.26 and spread 3.37, and BatchNorm brings them back to a narrow range. That is why its running variances are large (15 to 32 in the table above): the layer's raw outputs are big because the pooled vector is big. After the first ReLU 30% of the 256 units are silent, and after the second ReLU 48% of the 64 units are. The 12 logits are

$$z = (\text{-0.19, -1.32, -0.05, -0.99, -0.62, -0.04, -0.24, } \mathbf{2.40}\text{, -0.39, -1.27, -0.68, -0.72})$$

in the alphabetical class order of Section 2.2, with the maximum at class 7, **Mortgage**. The three largest contributions to that logit come from hidden units 46, 21 and 48 (0.235, 0.205, 0.185), plus the bias 0.060.

# 5 From logits to a loss

## 5.1 Softmax, computed stably

**General idea**

Softmax turns logits into probabilities, $s_k = e^{z_k} / \sum_j e^{z_j}$. It is unchanged by subtracting a constant from every logit, so subtracting the row maximum first keeps every exponent $\leq 0$ and avoids overflow, without changing the result.

```
    def forward(self):
        max_logits = self.logits.max(axis=1,keepdims=True)
        self.max_indices = self.logits.argmax(axis=1,keepdims=True)
        stable_logits = self.logits - max_logits
        e_logits = np.exp(stable_logits)
        e_logits_sum = e_logits.sum(axis=1,keepdims=True)
        self.probs = e_logits / e_logits_sum
        logprobs = np.log(self.probs)
        smoothing = 0.1
        n_classes = self.logits.shape[1]
        target_probs = np.full_like(self.probs, smoothing / n_classes)
        target_probs[np.arange(len(self.targets)), self.targets] += (1 - smoothing)

        loss = -(target_probs * logprobs).sum(axis=1).mean()
        self.B = len(self.targets)
        self.logprobs = logprobs
        self.target_probs = target_probs

        self.e_logits_sum = e_logits_sum
        self.e_logits = e_logits
        self.stable_logits = stable_logits
        return loss
```

src/crossentropy.py: stable softmax, label-smoothed targets, mean loss

## 5.2 Label smoothing

**General idea**

Plain cross-entropy compares the prediction with a one-hot target. Label smoothing with $\alpha = 0.1$ spreads a little mass over all classes:

$$q_k = \tfrac{\alpha}{K} + (1-\alpha)\,1[k=y] \quad \Rightarrow \quad q_y = 0.9083,\;\; q_{k\neq y} = 0.0083.$$

$$\ell = -\sum_{k=1}^{12} q_k \log s_k, \qquad \mathcal{L} = \tfrac{1}{B}\textstyle\sum_n \ell_n.$$

The model is never asked for 100% confidence, which tempers the logits. It also means the smallest achievable loss is the entropy of $q$ (about 0.53 nats), not 0, so **smoothed and unsmoothed losses cannot be compared**. This is why Phase 2 had to adopt the same smoothing before its losses meant anything next to Phase 1's.

## 5.3 The loss of the example

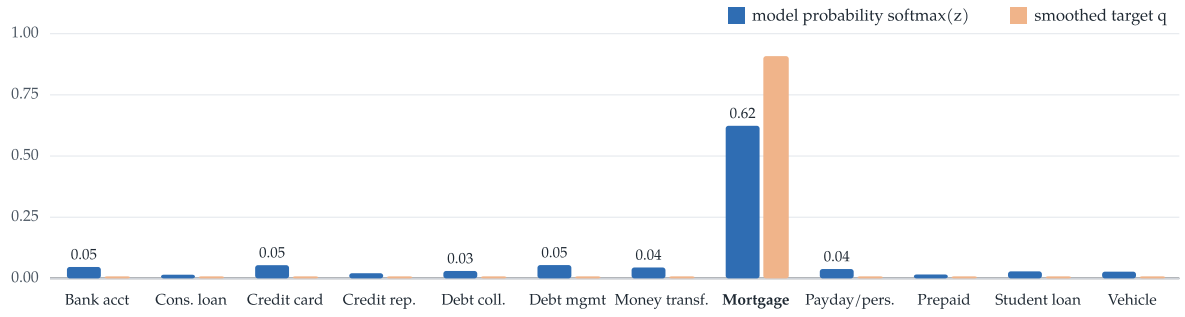



**Figure 9.** Softmax of the example's logits (blue) against the smoothed target (orange). The model gives Mortgage 0.624; the target asks for 0.908.

**In this repository**

With $s_{\text{Mortgage}} = 0.6236$ the smoothed loss is $\ell = 0.7465$ (computed by hand and by `CrossEntropyLoss`, identical), while the plain cross-entropy would be $-\log 0.624 = 0.4723$. The epoch-3 model's mean smoothed loss over the test set is 1.1757.

**In this repository: a confident, plausible 'error'**

The second example (test row 563, labelled **Consumer Loan**) reads: *"This car lot sold me a bad car, not even having the vehicle an entire year the engine went out and the company refused to fix it or put me in another vehicle at their cost."*. The model gives **Vehicle loan or lease 0.807**, Consumer Loan 0.073, Payday/title/personal loan 0.030. A human reader would also call this a vehicle complaint. That is the label-overlap problem of Section 9.4, on one example.

# 6 Backpropagation by hand

## 6.1 The rule that organises everything

**General idea**

The loss is one number, so its gradient with respect to any tensor $T$ has exactly the shape of $T$. For a module $Y = f(X)$, backpropagation receives $G_Y$ from the module after it and must return $G_X$ (and the gradients of its own parameters):

$$G_X[\alpha] = \sum_{\beta} \frac{\partial Y[\beta]}{\partial X[\alpha]}\, G_Y[\beta].$$

Nobody builds the full Jacobian. Each layer has a short hand-derived rule, and the network applies the rules in reverse order. This is exactly what `MLPHead.backward` does.

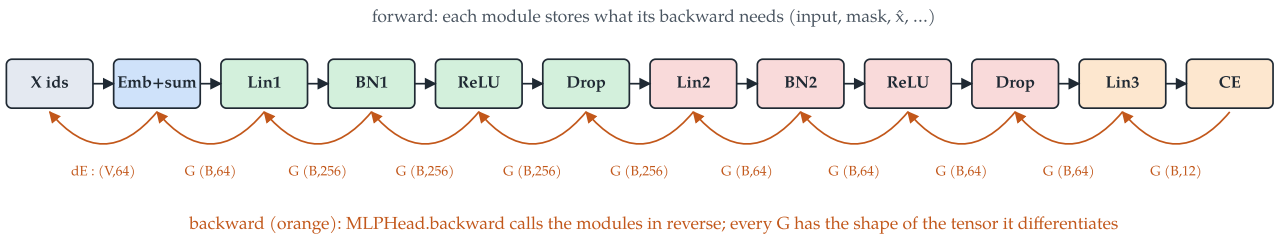



**Figure 10.** The forward chain (top) and the gradients flowing back (orange). Each forward call stores what its backward needs: Linear keeps its input, ReLU its mask, Dropout its mask, BatchNorm x̂ and the batch statistics.

```
49      def backward(self,dlogits):
50          g = self.linear3.backward(dlogits)
51          g = self.dropout2.backward(g)
52          g = self.relu2.backward(g)
53          g = self.bn2.backward(g)
54          g = self.linear2.backward(g)
55          g = self.dropout1.backward(g)
56          g = self.relu1.backward(g)
57          g = self.bn1.backward(g)
58          g = self.linear1.backward(g)
59          return g
```

src/mlp.py: the backward pass is the forward list, reversed

## 6.2 Softmax + cross-entropy: the gradient on the logits

**General idea**

For one example, $\ell = -\sum_k q_k \log s_k$ with $\log s_k = z_k - \log \sum_j e^{z_j}$. Since $\sum_k q_k = 1$,

$$\frac{\partial \ell}{\partial z_k} = s_k - q_k \qquad \Longrightarrow \qquad \boxed{G_Z = \tfrac{1}{B}\,(S - Q)} \in \mathbb{R}^{B\times 12}.$$

Predicted probability minus target, divided by the batch size. Each row sums to zero: the true class is pushed up by $q_y - s_y$ and every other class is pushed down by its excess probability.

**In this repository: the long way, checked against the short way**

`backward()` does not use the closed form. It walks back through every intermediate of the forward pass: log, the division by the sum, the exponential, and even the subtraction of the row maximum (lines 30 to 40). On a real training batch (32 complaints, trained model, train mode), its output equals $(S - Q)/B$ to **1.1e-17**, and the gradient that flows into the max-subtraction path is at most 1.8e-17 in absolute value, which is zero up to rounding: as it should be, since softmax ignores constant shifts.

```
29      def backward(self):
30          dlogprobs = -self.target_probs / self.B
31          # dlogprobs[np.arange(self.B),self.targets] = (-1/self.B)
32          dprobs = dlogprobs * (1/self.probs)
33          de_logits_sum = dprobs * self.e_logits * (-1/(self.e_logits_sum)**2)
34          de_logits_sum = de_logits_sum.sum(axis=1,keepdims=True)
35          de_logits = dprobs * 1/self.e_logits_sum + de_logits_sum
36          dstable_logits = de_logits * self.e_logits
37          dmax_logits = -dstable_logits.sum(axis=1,keepdims=True)
38          ph = np.zeros_like(dstable_logits)
39          ph[np.arange(self.B),self.max_indices.flatten()] = dmax_logits.flatten()
40          dlogits = dstable_logits + ph
41          return dlogits
```

For the first complaint of that batch (true class 8, *Payday/title/personal loan*, predicted 0.765) the gradient row is

(-0.00004, **+0.00361**, +0.00003, -0.00007, -0.00008, -0.00014, +0.00014, +0.00022, **-0.00449**, -0.00018, **+0.00112**, -0.00012)

The largest positive entry is class 1, *Consumer Loan* (predicted 0.124): the update will push that confusion down.

## 6.3 Linear layer

**General idea**

Forward $Y = XW + b$ with $X \in \mathbb{R}^{B \times d_{\text{in}}}$, $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$. Then

$$G_W = X^\top G_Y, \qquad G_b = \textstyle\sum_n G_Y[n, :], \qquad G_X = G_Y W^\top.$$

The rule: the gradient of a product with respect to one factor is the upstream gradient multiplied by the transpose of the other factor, on whichever side makes the shapes fit. The batch dimension disappears in $G_W$ and $G_b$: parameter gradients sum over all examples.

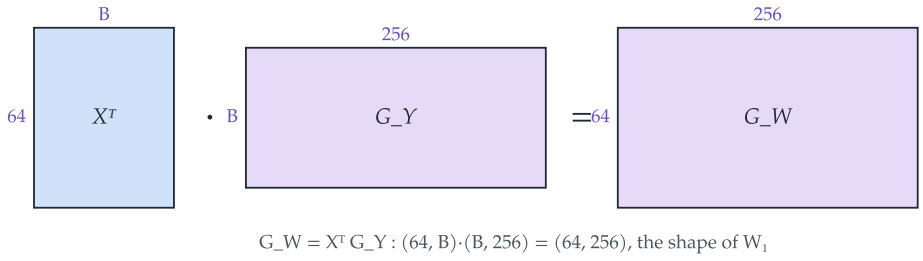
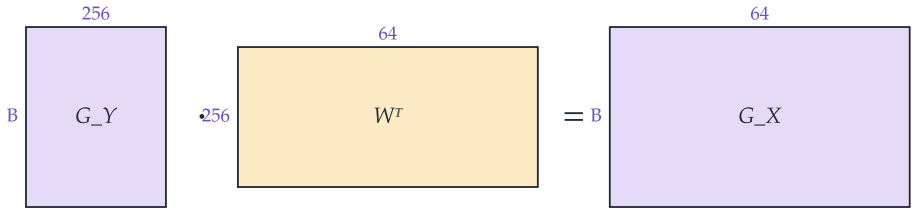



**Figure 11a.** Weight gradient of Linear1.




G_X = G_Y W$^T$ : (B, 256)·(256, 64) = (B, 64), the shape of X

**Figure 11b.** Input gradient of Linear1, passed on to the embedding.

```
18      def backward(self,local_grad):
19          self.local_grad = local_grad
20          dout = local_grad
21          self.dW = self.batch.transpose() @ dout
22          dX = dout @ self.weight.transpose()
23          if self.has_bias:
24              self.db = dout.sum(axis=0,keepdims=True)
25          return dX
```

src/linear.py: dW and db are stored on the module, dX is returned

## 6.4 Dropout and ReLU

**General idea**

Both are element-wise, so their backward passes are element-wise products with what the forward pass remembered: $G_X = G_Y \odot m/(1-p)$ for dropout (the **same** mask as in the forward pass) and $G_X = G_Y \odot \mathbb{1}[x > 0]$ for ReLU. A dropped or silent unit passes no gradient back, so its incoming weights get no update from this example.

## 6.5 Batch normalisation

**General idea**

In training, $\hat{x} = (x-\mu)/\sqrt{\sigma^2+\varepsilon}$ depends on $x$ directly *and* through $\mu$ and $\sigma^2$, which depend on every row of the batch. With $G_{\hat{x}} = G_Y \odot \gamma$:

$$G_\gamma = \textstyle\sum_n G_Y \odot \hat{x}, \quad G_\beta = \sum_n G_Y, \quad G_{\sigma^2} = \sum_n G_{\hat{x}}\,(x-\mu)\left(-\tfrac{1}{2}\right)(\sigma^2+\varepsilon)^{-3/2}, \quad G_\mu = -\sum_n \frac{G_{\hat{x}}}{\sqrt{\sigma^2+\varepsilon}},$$

$$G_X = \frac{G_{\hat{x}}}{\sqrt{\sigma^2+\varepsilon}} + G_{\sigma^2}\frac{2(x-\mu)}{B} + \frac{G_\mu}{B}.$$

```
29      def backward(self,local_grad):
30          self.grad = local_grad
31          dresult = self.grad
32          dx_norm = (dresult * self.gamma)
33          dgamma = (dresult * self.x_norm).sum(axis=0)
34          dbeta = dresult.sum(axis=0)
35
36
37          dloc_var = (dx_norm * (-(1/2)*(self.x-self.loc_mean)*(self.loc_var+self.eps)**(-3/2))).s
    um(axis=0,keepdims=True)
38          dloc_mean = (-dx_norm * (1/np.sqrt(self.loc_var + self.eps))).sum(axis=0,keepdims=True)
    #- dloc_var * (2/self.x.shape[0]) * (self.x - self.loc_mean)
39          dx = dx_norm * ((1/np.sqrt(self.loc_var + self.eps))) + dloc_var * (2/self.x.shape[0])*
    (self.x-self.loc_mean) + dloc_mean * (1/self.x.shape[0])
40          self.dgamma = dgamma
41          self.dbeta = dbeta
42          return dx
```

src/batchnorm.py: the backward pass follows these equations line by line

**Detail / deviation**

Textbook derivations also include, in $G_\mu$, a term through the variance: $G_{\sigma^2} \cdot \frac{-2}{B} \sum_n (x_n - \mu)$. Line 38 leaves it out (it is the commented code). That is correct, not a shortcut: $\sum_n (x_n - \mu) = 0$ by the definition of the mean, so the term is exactly zero. The gradient checks below confirm it.

## 6.6 The embedding: a scatter-add

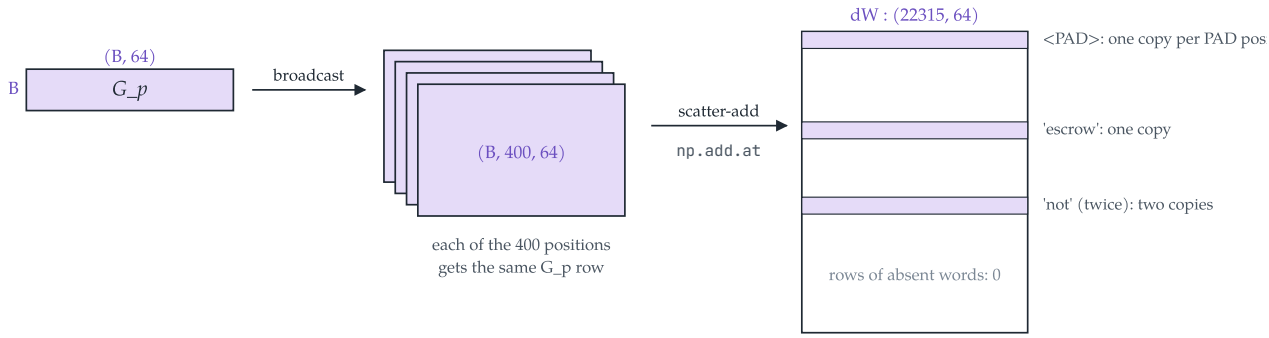



**Figure 12.** The sum over positions copies the gradient of p to each of the 400 rows that were summed. Those rows are then added into dW at their token IDs. Repeated tokens accumulate, and the <PAD> row receives one copy per padding position.

```
    def backward(self,local_grad):
        B,T = self.batch.shape
        dresult_summed = local_grad # B,C
        dresult = np.broadcast_to(dresult_summed[:,np.newaxis,:],(B,T,self.embed_dim)) # this ne
eds to be B,T,C
        dW = np.zeros_like(self.weight)
        # for i in range(B):
        #     for j in range(T):
        #         dW[self.batch[i,j],:] += dresult[i,j,:]
        np.add.at(
            dW,
            self.batch.reshape(-1),
            dresult.reshape(-1,self.embed_dim)
        )

        self.dW = dW
        return dW
```

src/embedding.py: np.add.at accumulates repeated indices; dW[ids] += ... would silently drop duplicates

**In this repository: which rows get a gradient**

On the real batch of Section 6.2, `dW` has 1,555 non-zero rows out of 22,315: the distinct tokens of 32 complaints. The batch contains 6,448 PAD positions, so row 0 collects 6,448 copies of the upstream gradient. Its gradient norm is **7.34**, the largest of all rows and about 1,600 times the median non-zero row (0.0045). Unlike the other layers, `Embedding.backward` returns `dW`, not an input gradient: it is the first layer, and token IDs cannot be differentiated.

## 6.7 One real step: the gradient of every parameter

| parameter | shape | ‖G‖ | ‖param‖ |
|---|---|---|---|
| W1 | (64, 256) | 0.1926 | 24.46 |
| gamma1 | (256,) | 0.0446 | 15.73 |
| beta1 | (256,) | 0.0757 | 2.69 |
| W2 | (256, 64) | 0.2509 | 15.78 |
| gamma2 | (64,) | 0.0804 | 7.38 |
| beta2 | (64,) | 0.1211 | 0.69 |
| W3 | (64, 12) | 0.4494 | 4.97 |
| b3 | (1, 12) | 0.0923 | 0.24 |
| embedding | (22315, 64) | 7.4055 | 95.11 |

Gradient norms on one training batch of 32 (loss 1.1675). Every gradient has its parameter's shape. The embedding gradient is dominated by the PAD row (norm 7.34 of 7.41).

## 6.8 Checking the hand-written gradients

**General idea**

A numerical gradient needs no calculus: nudge one number by $\pm\varepsilon$ and measure the change in a scalar output, $\partial L/\partial\theta \approx (L(\theta+\varepsilon) - L(\theta-\varepsilon))/2\varepsilon$, with error of order $\varepsilon^2$. The layer tests use the scalar $L = \sum Y \odot G$ for a random $G$, so that $\partial L/\partial Y = G$ exactly and `backward(G)` should reproduce the numerical result.

| Module test (`python -m src.<module>`) | max \|analytical − numerical\| |
|---|---|
| Linear (dX, dW, db at [0,0]) | 1.75e-11, 8.55e-12, 5.41e-11 |
| BatchNorm1d (all entries of dX, dγ, dβ) | 5.20e-11, 2.37e-11, 9.41e-12 |
| LayerNorm1d (all entries) | 1.33e-10, 2.12e-11, 2.15e-11 |
| Embedding (all entries of dW) | 9.59e-13 |

The per-layer checks in the repository, as last run on 30 Oct 2026.

**Added for this report: a whole-network check**

The layer checks test each rule in isolation. For this report the whole chain was also checked: on 8 real complaints, with BatchNorm in training mode and dropout switched off (so the loss is a deterministic function of the parameters), the full manual backward (cross-entropy → head → embedding) was compared with central differences of the real loss:

| entry | analytical | numerical ($\varepsilon = 10^{-5}$) | rel. error |
|---|---|---|---|
| W1[0,0] | -1.495587e-04 | -1.495587e-04 | 5.4e-09 |
| W1[5,17] | -8.615484e-04 | -8.615484e-04 | 8.9e-10 |
| W1[63,255] | +3.357496e-03 | +3.357496e-03 | 1.4e-09 |
| gamma1[3] | -2.619404e-03 | -2.619404e-03 | 4.3e-10 |
| gamma1[100] | -1.893309e-03 | -1.893309e-03 | 3.4e-10 |
| beta2[7] | +1.546310e-02 | +1.546310e-02 | 1.3e-10 |
| W2[10,3] | -2.289376e-04 | -2.289376e-04 | 5.9e-09 |
| W2[200,60] | +2.646746e-05 | +2.646746e-05 | 1.0e-08 |
| W3[0,0] | -1.449507e-03 | -1.449507e-03 | 1.3e-09 |
| W3[40,7] | +6.607674e-03 | +6.607674e-03 | 1.6e-10 |
| b3[0,2] | +2.314882e-02 | +2.314882e-02 | 1.1e-11 |
| embedding[4564,0] | +5.279916e-03 | +5.279916e-03 | 2.5e-10 |
| embedding[4564,33] | +6.835335e-03 | +6.835335e-03 | 5.7e-10 |
| embedding[0,5] (<PAD>) | -2.547568e-01 | -2.520820e-01 | 5.3e-03 |

All entries agree to $10^{-8}$ or better, except the PAD row at $\varepsilon = 10^{-5}$. That entry is used 1,771 times in the 8 complaints, so a nudge of $10^{-5}$ moves the pooled vectors by up to about 0.02 and crosses ReLU kinks: the finite difference, not the gradient, is inaccurate. With $\varepsilon = 10^{-6}$ the relative error is 2.3e-10.

# 7 Learning: AdamW and the training loop

## 7.1 AdamW

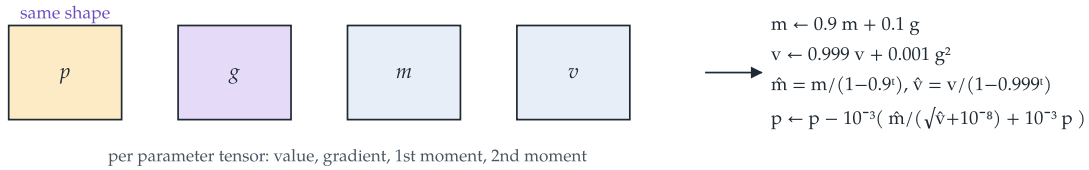



**Figure 13.** AdamW keeps two extra tensors per parameter, each with the parameter's shape: running averages of the gradient and of its square.

```
18      def step(self,grads):
19          self.t +=1
20          for i,(p,g,m,v) in enumerate(zip(self.params,grads,self.ms,self.vs)):
21              # m is m_{t-1}
22              m_t = self.beta1 * m + (1-self.beta1)*g
23              self.ms[i] = m_t
24              v_t = self.beta2 * v + (1-self.beta2)*g**2
25              self.vs[i] = v_t
26              v_hat = v_t/(1-self.beta2**self.t)
27              m_hat = m_t/(1-self.beta1**self.t)
28              p -= self.lr*((m_hat / (np.sqrt(v_hat) + self.eps))+   self.w_decay * p)
```

src/optimizer.py: one step for every parameter tensor; p -= ... updates the arrays in place

**General idea**

With step counter $t$, gradient $g$, $\beta_1 = 0.9$, $\beta_2 = 0.999$:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g^2, \qquad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

$$\theta \;\leftarrow\; \theta - \eta\Big(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} + \lambda\,\theta\Big), \qquad \eta = 10^{-3},\ \lambda = 10^{-3}.$$

Dividing by $\sqrt{\hat{v}}$ gives every parameter its own step size, roughly $\eta$ in units of its typical gradient. The bias correction undoes the zero initialisation of $m$ and $v$ in the first steps. The weight decay $\lambda\theta$ is *decoupled*: it is added to the update directly instead of to the gradient, so it is not rescaled by $\sqrt{\hat{v}}$. That is the difference between AdamW and Adam with L2 regularisation.

**In this repository: two small worked cases**

The repository's own test (`python -m src.optimizer`, constant gradient 0.1, $\eta = 0.1$) gives $\hat{m} = 0.1$ and $\hat{v} = 0.01$ at every step, so the parameter moves by 0.1 per step: $1 \to 0.9 \to 0.8 \to 0.7$. The tiny excess ($0.90000001$) is $\epsilon$ in the denominator.

On the real gradients of Section 6.7, a fresh optimizer's first step moves **every** entry of $W_3$ by between $0.99916{\cdot}10^{-3}$ and $1.00039{\cdot}10^{-3}$, whatever its gradient: at $t = 1$, $\hat{m}/\sqrt{\hat{v}} = g/|g| = \pm 1$. For example, gradients -0.00074, +0.00756, +0.00093 give updates of the same size, $1.0001{\cdot}10^{-3}$, with the opposite sign. Adam's first step is a sign step.

**Detail / deviation**

The embedding gradient is a dense `(22315, 64)` array that is zero for tokens not in the batch. AdamW still updates those rows: their momentum $m$ keeps moving them for a while after they were last seen, and weight decay shrinks all rows at every step. PyTorch's `nn.Embedding` with `torch.optim.AdamW` also produces dense gradients and behaves the same way, so this does not separate the two phases. Weight decay is applied to every parameter, including $\gamma$, $\beta$ and the biases, in both phases.

## 7.2 The training loop

```python
for e in range(epochs):
    epoch_loss_sum = 0
    perm = np.random.permutation(epoch_size)  # fresh shuffle order each epoch

    for i in range(epoch_size // batch_size):
        start = i * batch_size
        end = start + batch_size
        batched_indices = perm[start:end]

        Vocabed_text = Whole_Vocab[batched_indices]
        targets = Whole_targets[batched_indices]

        Embedded_text = Embeddor(Vocabed_text)  # (B, 400) → (B, 64), sum-pooled

        forward = NeuralNet(Embedded_text)  # (B, 64) → (B, 12) logits
        loss = CrossEntropyLoss(forward, targets=targets)
        loss_val = loss()
        epoch_loss_sum += loss_val

        dlogits = loss.backward()
        dEmbedded = NeuralNet.backward(dlogits=dlogits)
        dEmbedding = Embeddor.backward(dEmbedded)
        grads = NeuralNet.gradients() + [dEmbedding]
        optim.step(grads=grads)

        if i % 500 == 0:
            print(f"epoch {e} iter {i}: loss {float(loss_val):.4f}")

    train_losses.append(float(epoch_loss_sum / (epoch_size // batch_size)))
```

train_numpy.py: one epoch = a fresh permutation, 2,691 steps of 32, then validation

**In this repository**

86,134 // 32 = **2,691** steps per epoch; the last 22 complaints of each permutation are skipped, different ones every epoch. 30 epochs are 80,730 updates. After each epoch the model is switched to eval mode (`set_train_mode(False)`: BatchNorm uses its running statistics, dropout is off), the validation set is scored in chunks of 1,024, and the parameters with the lowest validation loss so far are copied. After the last epoch the best copy is restored and the test set is evaluated once.

## 7.3 Initialisation and the first loss

**In this repository: why the first loss is 3.78**

Re-running the start of `train_numpy.py` reproduces the first logged loss exactly: **3.7817**, well above $\ln 12 = 2.48$, the loss of uniform predictions. The reason is the initial scale of the logits. After BatchNorm and ReLU, $\mathbb{E}[h^2] \approx 0.5$; dropout's factor 2 makes it $\approx 1$; the He-normal $W_3$ has variance $2/64$ summed over 64 inputs, so $\mathrm{Var}(z) \approx 2$. Measured: **2.17** (std 1.47). Random logits of variance $\sigma^2$ cost about $\ln K + \sigma^2/2 \approx 3.5$ nats. The first few hundred steps are spent shrinking these overconfident random scores: the hockey stick at the start of the Phase 1 curve.

At initialisation the pooled vectors are also dominated by padding: every row of $E$ is $\mathrm{N}(0, 0.01^2)$, but the PAD row is added about 250 times with the *same* sign, so it contributes about $0.01 \cdot 250 = 2.5$ per dimension, against $0.01\sqrt{150} \approx 0.12$ from the word rows. Measured std of the pooled values on the first batch: 2.02.

# 8 NumPy versus PyTorch: what autograd replaces

## 8.1 The same network in about thirty lines

```
class MLPHead(nn.Module):
    def __init__(self,vocab_size,n_embd,input_dim,output_dim):
        super().__init__()
        self.embedding_layer = nn.Embedding(vocab_size,n_embd)
        nn.init.normal_(self.embedding_layer.weight, std=0.01)  # match Phase 1
        self.linear1 = nn.Linear(input_dim,4*input_dim,bias=False)
        self.bn1 = nn.BatchNorm1d(4*input_dim,momentum=0.3)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=0.5)
        self.linear2 = nn.Linear(4*input_dim,input_dim,bias=False)
        self.bn2 = nn.BatchNorm1d(input_dim,momentum=0.3)
        self.dropout2 = nn.Dropout(p=0.5)
        self.linear3 = nn.Linear(input_dim,output_dim)
        self.sequence = nn.Sequential(self.linear1,
                            self.bn1,
                            self.relu,
                            self.dropout,
                            self.linear2,
                            self.bn2,
                            self.relu,
                            self.dropout2,
                            self.linear3)

    def forward(self,x):
        embedded_x = self.embedding_layer(x).sum(axis=1)
        result = self.sequence(embedded_x)
        return result
```

train_torch.py: the whole Phase 2 model. There is no backward method anywhere.

**General idea**

Autograd records every operation of the forward pass in a graph, and `loss.backward()` walks that graph in reverse, applying each operation's derivative rule. These are the same rules as in Chapter 6, written once inside the library instead of once per layer by hand. Everything that Phase 1 does manually (storing inputs and masks, ordering the backward calls, accumulating repeated embedding rows, summing gradients over the batch) is done by the framework. Phase 2 is therefore an independent implementation of the gradient: if the hand-written backward passes had a sign error or a missing term, the two phases would train differently.

```
        Embedded_text = Embeddor(Vocabed_text)  # (B, 400) → (B, 64), sum-pooled

        forward = NeuralNet(Embedded_text)  # (B, 64) → (B, 12) logits
        loss = CrossEntropyLoss(forward, targets=targets)
        loss_val = loss()
        epoch_loss_sum += loss_val

        dlogits = loss.backward()
        dEmbedded = NeuralNet.backward(dlogits=dlogits)
        dEmbedding = Embeddor.backward(dEmbedded)
        grads = NeuralNet.gradients() + [dEmbedding]
        optim.step(grads=grads)
```

Phase 1: forward, manual backward, manual update

```
        xb = Whole_vocab[idx].to(device)
        targets = whole_targets[idx].to(device)
        forward = model(xb)
        loss = F.cross_entropy(forward,target=targets,label_smoothing=0.1)
        epoch_loss_sum += loss.item()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Phase 2: forward, autograd, library update

## 8.2 Conventions, side by side

| | Phase 1 (NumPy) | Phase 2 (PyTorch) | Status |
|---|---|---|---|
| Linear weight | `(in, out)`, `x @ W` | `(out, in)`, `x @ W.T` | layout only |
| Linear bias | `(1, out)` | `(out,)` | layout only |
| Linear init | He normal, std $\sqrt{(2/\text{in})}$ | Kaiming uniform ($a = \sqrt{5}$) | **not matched** |
| Embedding init | $N(0, 0.01^2)$ | `nn.init.normal_(std=0.01)` | matched |
| BatchNorm momentum | 0.7 = weight of the *old* value | `momentum=0.3` = weight of the *new* value | matched ($1 - 0.7$) |
| BatchNorm running var | biased ($/B$) | unbiased ($/(B-1)$) for the running stat | **not matched** (factor $32/31$) |
| Dropout | $p = 0.5$, inverted | `nn.Dropout(p=0.5)` | matched |
| Loss | own CE, smoothing 0.1 | `F.cross_entropy(label_smoothing=0.1)` | matched |
| Optimizer | own AdamW | `torch.optim.AdamW`, same hyperparameters | matched |
| Gradients | hand-written `backward` | autograd (`loss.backward()`) | the point of the comparison |
| Random numbers | NumPy, seed 42 | PyTorch, seed 42 | different streams: batch order and masks differ |
| Checkpoint | copy of `parameters()` + BN stats | `copy.deepcopy(state_dict())` | matched after the fix |

**Detail: why 0.3, and what was changed**

The momentum conventions are opposite. Phase 1 writes `r = 0.7·r + 0.3·batch`; PyTorch writes `r = (1 - m)·r + m·batch`. The same average therefore needs `momentum=0.3` in PyTorch; `momentum=0.7` would have put 70% of the weight on each new batch. Before the revision Phase 2 also differed in dropout (0.3), label smoothing (none), BatchNorm momentum (the default 0.1) and embedding initialisation ($N(0, 1)$); with $N(0, 1)$ and 400 summed positions, its first-epoch validation accuracy was 54.2%. Matching those four settings raised it to 72.9%.

## 8.3 What "the two match" means in this project

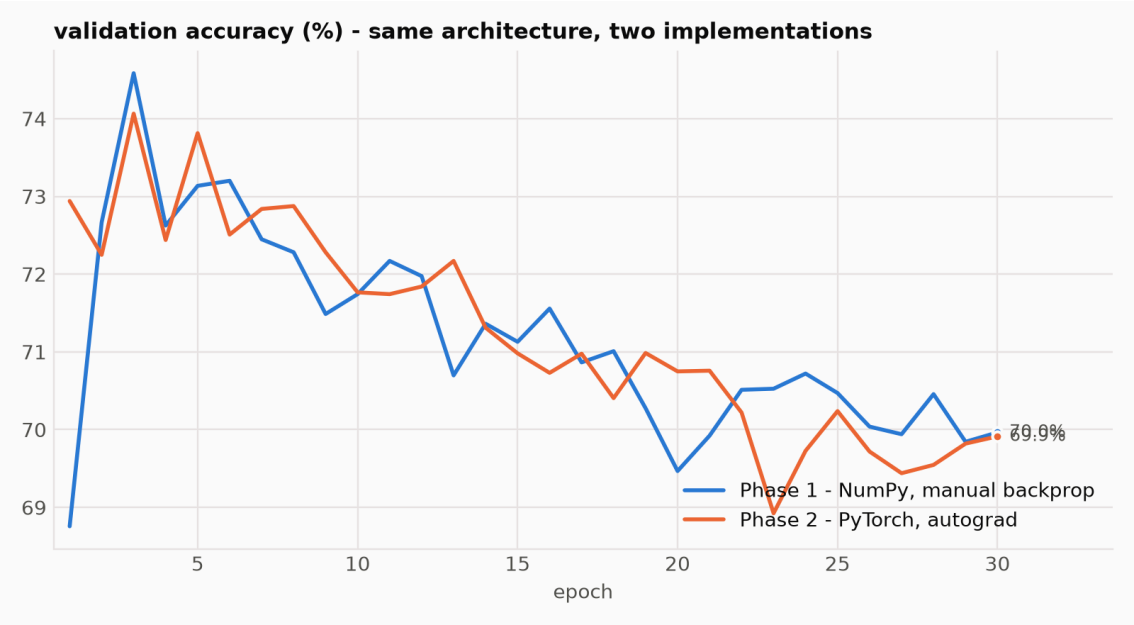



**Figure 14.** Validation accuracy per epoch, Phase 1 (blue) and Phase 2 (orange), from `results/*/metrics.json`.

| epoch | P1 val loss | P1 val acc | P2 val loss | P2 val acc |
|---|---|---|---|---|
| 1 | 1.3023 | 68.76 | 1.2236 | 72.94 |
| 2 | 1.2141 | 72.67 | 1.2235 | 72.25 |
| **3** | **1.1760** | **74.58** | **1.1781** | **74.06** |
| 4 | 1.2049 | 72.62 | 1.2061 | 72.44 |
| 5 | 1.2017 | 73.14 | 1.1807 | 73.81 |
| 6 | 1.1958 | 73.20 | 1.2119 | 72.51 |
| 30 | 1.3641 | 69.96 | 1.3524 | 69.91 |

Validation loss (label-smoothed) and accuracy (%) per epoch. The best epoch (lowest validation loss) is in bold.

**In this repository**

"Match" means three observable things, all from the saved results:

1. **Same trajectory after the first epoch.** Both phases reach their lowest validation loss at **epoch 3** (1.1760 vs 1.1781, a difference of 0.002) and then overfit the same way: validation loss rises to about 1.36 by epoch 30 while training loss falls to 0.874 in both.
2. **Same held-out result.** Test accuracy 74.54% vs 74.10%, macro F1 0.729 vs 0.726, test loss 1.1757 vs 1.1778.
3. **Same failure pattern.** The same two classes are the worst in both (Section 9.4).

It does **not** mean identical numbers. The random streams differ, so the batches and dropout masks differ, and two settings are not matched. With a single seed per phase, differences below about one point are within run-to-run noise. The proof that each gradient rule is right is the gradient checks of Section 6.8; this comparison shows that the rules, assembled into a training loop, learn what a reference implementation learns.

**Detail / deviation**

The one visible gap is epoch 1: 68.8% (Phase 1) against 72.9% (Phase 2). It comes from the unmatched Linear initialisation. Phase 1's He-normal output layer starts with logit variance about 2 (first loss 3.78, Section 7.3); PyTorch's Kaiming-uniform starts much closer to uniform predictions (first loss 2.58). Phase 1 spends part of its first epoch undoing that, and from epoch 2 the curves overlap.

# 9 Evaluation and results

## 9.1 Metrics

**General idea**

For class $k$: **precision** = of the complaints predicted $k$, the fraction that are $k$; **recall** = of the complaints that are $k$, the fraction predicted $k$; **F1** = their harmonic mean. **Macro F1** averages F1 over the 12 classes with equal weight, so small classes count as much as large ones; **accuracy** is the fraction of all complaints classified correctly. The test split is used **once**, after every choice (hyperparameters, checkpoint) was made on validation, because each look at a score that guides a decision fits that set's quirks.

## 9.2 Headline results

| Model | Test accuracy | Macro F1 | Best epoch |
|---|---|---|---|
| **Phase 1:** MLP, NumPy, hand-written backprop | **74.54%** | **0.729** | 3 of 30 |
| **Phase 2:** same MLP, PyTorch autograd | 74.10% | 0.726 | 3 of 30 |
| Baseline: TF-IDF (unigrams) + logistic regression | 77.24% | 0.758 | n/a (C = 1) |

Held-out test split, 26,917 complaints. One run per model, seed 42.

## 9.3 Training curves

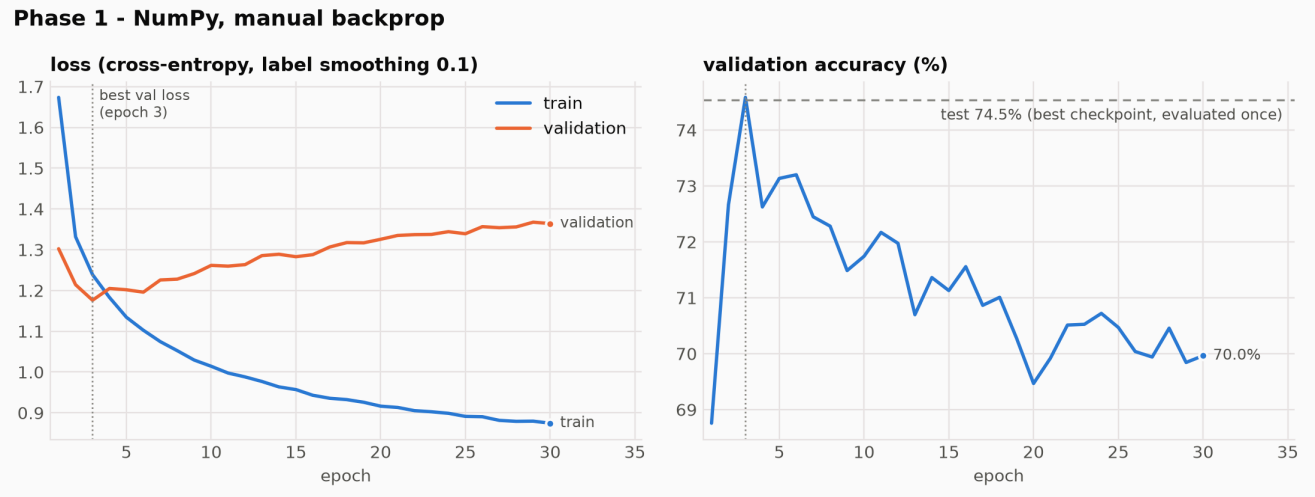



**Figure 15.** Phase 1, from `results/numpy/metrics.json`. Validation loss is lowest at epoch 3 (dotted line); afterwards training loss keeps falling and validation loss rises: the model memorises.

**In this repository**

For the first epochs validation loss is *below* training loss. That is not a bug: training loss is averaged over an epoch with dropout active and with weights that are still improving, while validation runs at the end of the epoch with dropout off. The curves cross around epoch 3, which is also where the checkpoint is taken.

## 9.4 Per-class behaviour and the label overlap

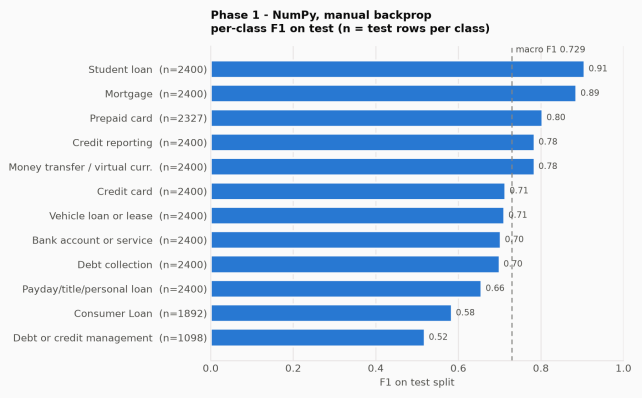



**Figure 16.** Phase 1 test F1 per class.

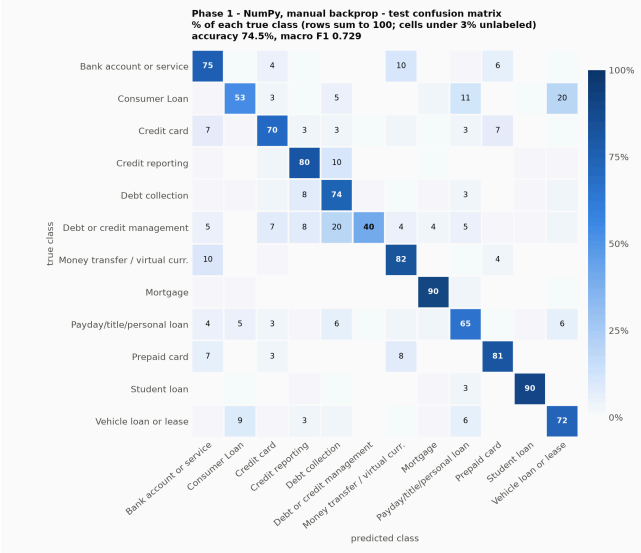



**Figure 17.** Phase 1 confusion matrix, % of each true class.

| Class | Recall P1 | Recall P2 | Recall TF-IDF | F1 P1 | F1 TF-IDF |
|---|---|---|---|---|---|
| Bank account or service | 0.75 | 0.75 | 0.78 | 0.70 | 0.76 |
| Consumer Loan | 0.53 | 0.56 | 0.58 | 0.58 | 0.62 |
| Credit card | 0.70 | 0.73 | 0.74 | 0.71 | 0.75 |
| Credit reporting | 0.80 | 0.81 | 0.84 | 0.78 | 0.80 |
| Debt collection | 0.74 | 0.74 | 0.75 | 0.70 | 0.72 |
| Debt or credit management | 0.40 | 0.37 | 0.46 | 0.52 | 0.56 |
| Money transfer / virtual currency | 0.82 | 0.75 | 0.82 | 0.78 | 0.82 |
| Mortgage | 0.91 | 0.87 | 0.92 | 0.89 | 0.92 |
| Payday/title/personal loan | 0.65 | 0.68 | 0.71 | 0.66 | 0.69 |
| Prepaid card | 0.81 | 0.83 | 0.83 | 0.80 | 0.82 |
| Student loan | 0.90 | 0.90 | 0.90 | 0.91 | 0.92 |
| Vehicle loan or lease | 0.72 | 0.68 | 0.73 | 0.71 | 0.73 |

**In this repository: the same two classes, three models**

Two classes are the weakest in **all three models**: *Debt or credit management* (recall 0.40 / 0.37 / 0.46) and *Consumer Loan* (0.53 / 0.56 / 0.58). In Phase 1, 19.6% of Consumer Loan test complaints are predicted as *Vehicle loan or lease* and 11.5% as *Payday/title/personal loan*, the newer categories that cover the same products (example 2 in Section 5.3 is one of them). Debt or credit management loses 19.7% to *Debt collection*, then 8.0% to *Credit reporting*, 7.3% to *Credit card* and smaller shares elsewhere, consistent with a broad catch-all label. Three very different models failing on the same two classes points at the labels rather than at any one model: when categories overlap by definition, part of the ~75% ceiling is label ambiguity.

## 9.5 The TF-IDF + logistic regression baseline

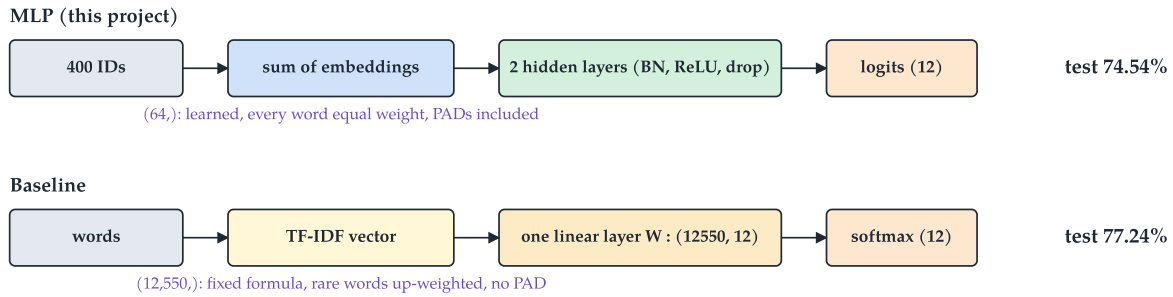



**Figure 18.** Both models see only which words occur, not their order. They differ in how the words become a vector.

**General idea**

TF-IDF gives each word $w$ of a document a weight $(1 + \log \mathrm{tf}_w) \cdot \mathrm{idf}_w$ (with `sublinear_tf=True`), where $\mathrm{idf}_w = \ln \frac{1+N}{1+\mathrm{df}_w} + 1$ is large for rare words; each document vector is then scaled to unit length. Logistic regression is a single linear layer with softmax, $s = \mathrm{softmax}(xW + b)$ with $W \in \mathbb{R}^{12550 \times 12}$: the same as the MLP's last layer, with no hidden layers and a fixed, not learned, input representation.

**In this repository**

`baseline_tfidf.py` fits the vectoriser on train only (`min_df=10`: 12,550 terms) and tries C ∈ {0.1, 1, 10} (inverse L2 strength) on validation: 74.13%, **76.61%**, 75.27%. The best value sits inside the grid. Test accuracy with C = 1 is **77.24%**, macro F1 0.758. For the Mortgage example the largest TF-IDF weights are *shortage* 0.32, *hundreds* 0.31, *penalties* 0.29, *thousands* 0.28, and the smallest are *that*, *on*, *xxxx*, *my* (about 0.06): common words are pushed down automatically (idf of *the* = 1.10, of *escrow* = 4.90).

**Detail / deviation**

Why the MLP loses by 2.7 points: its sum gives every word the same footing (common words and `xxxx` are never down-weighted), squeezes the complaint into 64 numbers instead of 12,550, adds hundreds of PAD rows (Section 3.5), and keeps punctuation-attached duplicates as separate words. None of these is a backpropagation problem: they are properties of the representation, which is what a v2 would change (Chapter 10).

## 9.6 The checkpoint bug

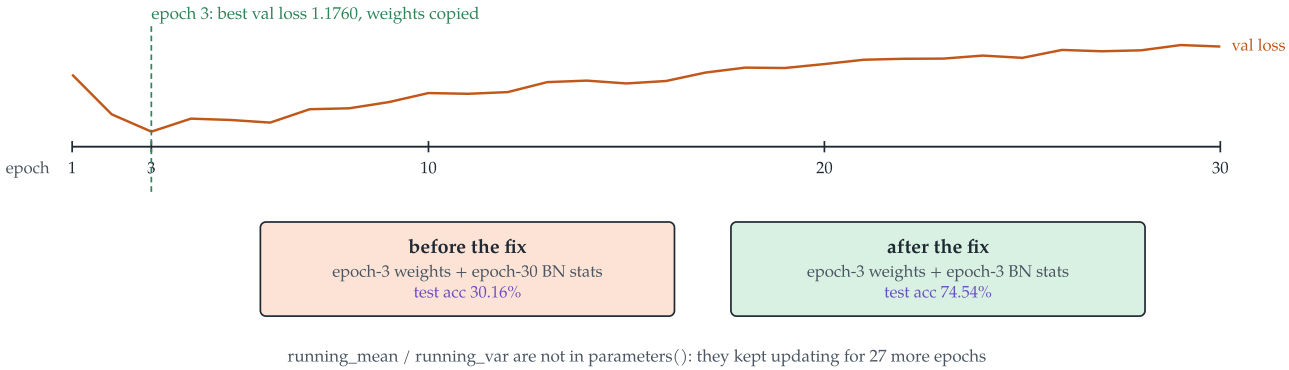



**Figure 19.** What the first 30-epoch run evaluated. The weights came from epoch 3, but BatchNorm's running statistics kept changing until epoch 30.

**In this repository**

The first version of the restore step copied `NeuralNet.parameters() + Embeddor.parameters()` and nothing else. BatchNorm's `running_mean` and `running_var` are not parameters, so after restoring, the epoch-3 weights were normalised with statistics accumulated by the epoch-30 weights. The first full run printed a validation loss of 1.1760 for its checkpoint and then a test accuracy of **30.16%** (macro F1 0.228), with almost every complaint sent to a few classes (Student loan recall 0.996). Every earlier check had passed: gradient checks test the maths of one layer, not the training procedure, and short runs could not show the bug because with one epoch the best epoch is also the last.

```
118     if current_val_loss < best_val:
119         best_val = current_val_loss
120         best_params = [p.copy() for p in NeuralNet.parameters() + Embeddor.parameters()]
121         best_epoch = e + 1
122         best_bn_stats = [(bn.running_mean.copy(), bn.running_var.copy())
123                 for bn in (NeuralNet.bn1, NeuralNet.bn2)]
124     set_train_mode(True)
125 
126 # Restore whichever epoch had the best val loss, not just the last one
127 all_params = NeuralNet.parameters() + Embeddor.parameters()
128 for p, best_p in zip(all_params, best_params):
129     p[...] = best_p
130 for bn, (mean, var) in zip((NeuralNet.bn1, NeuralNet.bn2), best_bn_stats):
131     bn.running_mean, bn.running_var = mean, var
132 
133 
134 print(f"best val loss (restored): {best_val}")
135 set_train_mode(False)
136 restored_val = float(CrossEntropyLoss(predict_logits(Vocabed_val), targets=target)())
137 print(f"restored val loss = {restored_val:.6f} (recorded best = {float(best_val):.6f})")
138 assert abs(restored_val - float(best_val)) < 1e-6, "restored checkpoint does not reproduce its v
    al loss"
```

train_numpy.py: the fix (lines 122-123 and 130-131) and the guard (136-138)

**General idea**

The general lesson: a restored checkpoint must reproduce the score it was saved with. The assertion on line 138 checks exactly that on validation data, without touching the test set, so it would have caught this bug on its own. PyTorch avoids it by design: `state_dict()` contains BatchNorm's running statistics as *buffers*.

# 10 Findings, limitations and conclusion

## 10.1 What the project shows

1. **The hand-written network is correct.** Every layer passes a numerical gradient check (errors around $10^{-10}$), the whole chain passes one too, and an independent PyTorch implementation trained with autograd follows the same validation trajectory and reaches the same test result (74.54% vs 74.10%).
2. **The weak classes are partly a labelling problem.** The same two categories are the worst in three different models, and their errors go to the categories that later replaced or overlap them.
3. **The model saturates after three epochs.** Both phases reach their best validation loss at epoch 3 and overfit afterwards, consistent with a bag-of-words representation that is exhausted early.
4. **Correct gradients do not make a correct training procedure.** The checkpoint bug passed every gradient check and was caught by the held-out evaluation; the script now asserts that the restored model reproduces its recorded validation loss.
5. **A linear model on TF-IDF features scores higher (77.24%).** The gap comes from the representation (equal-weight sum, padding, 64 dimensions), not from the learning machinery.

## 10.2 Limitations

- **Bag of words only:** no word order, no sequence model, no attention, no pretrained embeddings.
- **Padding is not masked:** the PAD row is summed hundreds of times and carries length information (Section 3.5).
- **Whitespace tokenisation** keeps punctuation attached to words.
- **One seed per configuration:** no variance estimate; differences below about one point are not meaningful.
- Some Phase 1 hyperparameters (dropout, BatchNorm momentum, label smoothing) are hard-coded inside `src/` instead of passed in, and two Phase 1/Phase 2 settings (Linear initialisation, biased running variance) are not matched.

## 10.3 Next experiments (a possible v2)

Mask the padding, or mean-pool over real tokens only; try TF-IDF-weighted pooling of the embeddings; strip punctuation in the tokenizer. Each would be tuned on validation and reported next to the v1 numbers and the baseline, whether or not it wins.

## 10.4 Conclusion

The project builds a text classifier from its smallest pieces: an integer ID per word, a table of 64-number vectors, three matrix products with normalisation and noise in between, a loss that compares 12 probabilities with a softened target, and the chain rule applied by hand in reverse. A second implementation that lets a library do the chain rule learns the same thing, which is the strongest evidence that the hand-written gradients are right. The honest result is a well-understood 74.5%: below a simple linear baseline, with the reasons identified (the representation and the labels), and with one real bug found and fixed along the way. Those reasons are also the plan for the next version.

# A Shape cheat sheet

| tensor | shape | note |
|---|---|---|
| token IDs (one complaint) | (400,) | int, PAD = 0, UNK = 1 |
| batch X | (32, 400) | `Whole_Vocab[idx]` |
| embedding table E | (22315, 64) | $N(0, 0.01^2)$ |
| gathered rows E[X] | (32, 400, 64) | temporary |
| pooled P | (32, 64) | sum over axis -2 |
| $W_1$ / $H_1$ | (64, 256) / (32, 256) | no bias |
| BN1 $\gamma$, $\beta$, running stats | (256,), (256,), (1, 256)×2 | momentum 0.7 |
| $W_2$ / $H_2$ | (256, 64) / (32, 64) | no bias |
| BN2 $\gamma$, $\beta$, running stats | (64,), (64,), (1, 64)×2 | |
| $W_3$, b / Z | (64, 12), (1, 12) / (32, 12) | logits |
| S, Q, G_Z | (32, 12) | $G_Z = (S - Q)/32$ |
| loss | scalar | mean over batch |
| $G_W_1$, $G_W_2$, $G_W_3$ | shape of W | $X^T G$ |
| G_P (into embedding) | (32, 64) | $G\ W^T$ |
| dE | (22315, 64) | scatter-add, PAD row largest |
| AdamW m, v | shape of each parameter | $\beta = (0.9, 0.999)$ |

# B How this document was verified

- **Code.** Every listing is copied from the repository with its real line numbers (`src/`, `train_numpy.py`, `train_torch.py`, `src/prepare_data.py`).
- **Results.** Accuracies, F1 scores, per-class recalls, confusion percentages and training histories are read from `results/numpy/metrics.json`, `results/torch/metrics.json` and `results/baseline_tfidf.json`; Figures 14 to 17 are the repository's own PNGs.
- **Worked examples.** The repository does not save model weights, so the published Phase 1 checkpoint was reproduced: `train_numpy.py --epochs 3` with the same seed and data gives the same logged losses at every printed step, best validation loss 1.1760259421754 (the published run: 1.176025942181673, equal to $10^{-11}$), and the same test accuracy (74.54%) and macro F1 (0.7294). Because the best epoch is 3 in the 30-epoch run, this is the same model. All per-example numbers (tokens, IDs, the PAD split, activations, logits, probabilities, gradients, the AdamW step, the whole-network check, the first loss) were computed with the repository's own `src/` modules on that checkpoint.
- **Data statistics** (lengths, padding share, UNK rate, class totals) were computed on `data/processed/*.csv` with `src/vocab.py`.
- **Gradient checks.** The per-layer numbers are the outputs of `python -m src.linear`, `src.batchnorm`, `src.layernorm` and `src.embedding`; the whole-network check (Section 6.8) was run for this report and is not part of the repository.
- **Not covered** because the project does not contain it: sequence models, attention, pretrained embeddings, deployment, and the CuPy path's own results (the published numbers come from the NumPy path).